Project Phase 3: Testing

CMPT 276: Introduction to Software Engineering
Simon Fraser University

Group 1

Mohamad Osman	301590514
Brian Chan	301439177
Keith	301625387
Daniil Bugakov	301540700
Dev Tomar	301548576

March 24, 2026

Contents

1 Overview **1**

2 Unit Testing **1**

3 Integration Testing **3**

4 Test Quality and Coverage **4**
4.1 Test Quality Measures 4
4.2 Line and Branch Coverage 4
4.3 Uncovered Code and Justification 5

5 Findings & Improvements **6**
5.1 Production Code Changes 6
5.2 Bugs and Observations 6

1 Overview

The goal of Phase 3 was to write unit and integration tests for Bank Heist, measure code coverage, and improve the production code where testing revealed issues. All 215 tests were written using JUnit 5 and are located under `src/test/java`. Coverage was measured with JaCoCo 0.8.12 and is generated automatically by running `mvn clean test`. The full JaCoCo report is available at `target/site/jacoco/index.html` after a build. During the testing phase, several bugs were discovered and fixed, and accessor methods were added to support test isolation. No external testing libraries beyond JUnit 5 and JaCoCo were used.

2 Unit Testing

Unit tests target individual classes in isolation. Each feature was identified, tested independently, and the corresponding test class is noted below.

- **CashStack** – Verifies that collecting a CashStack increases the player's balance by 100 and decreases the remaining static money count.
- **Guard** – Verifies that the Guard is created at the correct position and that update moves the Guard toward the Thief over successive calls.
- **GlueTrap** – Verifies that the trap is instantiated correctly, updates its isCollected state when triggered, and correctly applies a 1-tick freeze to the board's standard stuckCount.
- **InkDart** – Verifies that balance goes down by 150 when triggered.
- **RareDiamond** – Verifies that collecting the bonus item adds 500 to the balance.
- **MovingEntity** – Verifies that moveTo correctly updates the entity's position and that canMoveTo returns false for positions outside the board and true for valid positions.
- **Position** – Verifies that coordinates are stored correctly, add returns a shifted position, and distanceTo calculates the correct distance between two positions.
- **PlaytimeManager** – Verifies that start() begins a session, pause() accumulates elapsed time, and reset() clears all state. Covers getElapsed() after pause, and formatTime() across nine boundary cases (zero, seconds, mins, rollover). 23 tests – 100% line and branch coverage.
- **Interface (state & rendering)** – Verifies all six GameState transitions (MENU, PLAYING, PAUSED, WIN, GAMEOVER, TUTORIAL), paintComponent rendering paths for each state, and panel dimensions. Confirms the board is paused when the game is not in the PLAYING state. 26 tests – 99% line coverage (3 instructions unreachable: System.exit(0) lambdas).

- **Interface (UI components)** – Verifies UIElement rendering under all four FitMode values (NATURAL, FIT, FILL, STRETCH), UIButton hit-testing, tick animation, and click handling. Covers null-image fallback drawing for both UIElement and UIButton, including the hovered placeholder path. 36 tests – UIButton 100% instruction and branch coverage.

- **Interface (interaction & state)** – Verifies button counts per game state (MAIN_MENU 3, PLAYING 1, PAUSED 5, WIN 3, GAME_OVER 3, TUTORIAL 1). Covers ESC key binding toggling PLAYING to PAUSED and back, returning from TUTORIAL to the remembered state, and no-op on MAIN_MENU/WIN/GAME_OVER. Confirms mouse press sets button pressed state, mouse release on-button triggers the action, mouse release off-button cancels, and mouseMoved sets/clears hover state. Covers playAgain (resets board, sets PLAYING), returnToMainMenu (resets board, sets MAIN_MENU), openTutorial (remembers return state), and tickAnimations. Mouse listener instructions 8% to 97%, ESC binding 16% to 100%.

- **Cell** – Verifies that a Cell correctly stores its image, position and its size. Also verifies that setImage() and setInteractable() update the state correctly. Tests different hasInteractable() cases: no item exists, item exists and has not been collected yet, item has been collected.

- **BoardFactory** – Verifies that makeBoard returns a valid Board for existing levels and throws an exception for invalid levels. Tests that the correct board layout is created by checking wall and entity (thief and guard) positions. Also verifies that Board follows the singleton pattern by ensuring multiple calls return the same instance.

- **Board (balance, freeze, visuals)** – Verifies getBalance/addBalance, setPaused, setInterface, singleton resetInstance, and applyFreeze/getStuckCount. Covers the VisualState inner class (constructor, setInitialPosition, startDash, update) and diamond spawn logic via reflection. Confirms a negative balance triggers GAMEOVER when an Interface is connected. 34 tests – 100% line coverage on all tested methods.

- **Board (movement & input)** – Verifies moveThief rotation angles for all four directions (up sets 0 degrees, down sets 180 degrees, left sets 270 degrees, right sets 90 degrees), no-op behaviour when paused or with zero input, stuck decrement blocking movement, wall collision rejection, and guard collision triggering GAME_OVER when an Interface is wired. Confirms WASD key binding actions set inputRow/inputCol correctly via ActionMap dispatch. Covers spawnRandomDiamond no-op when paused or when a diamond is already active, gameMovement timer tick while paused, and drawEntityWithTrail mid-dash rendering (progress less than 1.0). 32 tests – Board instructions 92% to 97%, Board branches 72% to 80%, key binding anonymous classes 40% to 100%.

- **Board (spatial queries)** – Verifies isInside for valid positions, negative row/column, and beyond-boundary values. Covers isWall on a known wall cell and a known empty cell. Confirms isOccupied returns true for the thief cell, true for the guard cell, and false for an empty cell. Covers decrementStaticMoney counting down to zero,

getStuckCount/getGuardStuckCount initial values, and resetInstance allowing a fresh singleton.

3 Integration Testing

Integration tests verify the interactions between multiple components of the system. Each test wires real instances of the involved classes together using custom board layouts rather than production levels, ensuring full control over entity and item placement.

- **ItemCollectionIntegrationTest** – Verifies that when the Thief moves onto an interactable cell, the Board correctly detects the collision, triggers the item effect, and updates the global balance. Tests the interaction between Thief, Board, and Interactable items. Uses a custom board layout instead of random spawning.
- **WallCollisionIntegrationTest** – Verifies that the Thief's coordinates remain the same after an illegal move attempt into a wall. Tests the interaction between Board and Thief movement logic. Uses a custom board where the Thief is spawned next to a wall and attempts to move into it.
- **GlueTrapIntegrationTest** – Verifies that the Thief receives a 1-tick penalty, while the Guard receives a 3-tick penalty. Uses a custom map that forces the Guard's pathfinding to walk into the trap. Tests the interaction between Board, GlueTrap, Thief, and Guard.
- **GuardIntegrationTest** – Verifies that when the Guard reaches the Thief's position, the game state updates to GAME_OVER. Also verifies the edge case where both the Guard and Thief occupy the exit cell at the same time, ensuring the loss condition takes priority over the win condition. Tests the interaction between Guard movement logic and Interface game state handling.
- **Exit condition** – Creates a real Board and Interface connected together. Teleports the thief one cell left of the exit and moves it in. Verifies the game stays in PLAYING when money remains (tested at counts 1 and 3), transitions to WIN when all money is collected, correctly transitions after the last bag is collected mid session, and reaches WIN regardless of balance when staticMoneyCount is 0. 5 tests.
- **Glue trap (thief)** – Steps the thief onto the glue trap on the test map and confirms stuckCount increments to 1. Verifies the thief cannot move while frozen and that stuckCount decrements to 0 after one blocked tick (freeze expires after exactly one tick). 3 tests.
- **Glue trap (guard)** – Confirms guardStuckCount is initialised to 0 and is independent of the thief's counter. Fires one manual timer tick on a board where the guard walks onto a glue cell and asserts guardStuckCount becomes 3. Fires three successive ticks on a board pre-set to guardStuckCount = 3 and asserts the counter decrements through 2 to 1 to 0. 3 tests.

4 Test Quality and Coverage

4.1 Test Quality Measures

The following measures were taken to ensure the quality of the test suite:

- Every test contains at least one meaningful assertion that checks a specific expected outcome rather than simply asserting that no exception is thrown.
- Each test method tests a single behaviour or condition. Test names describe the scenario being verified so that a failure message immediately identifies what broke.
- Tests are independent and repeatable. The Board singleton is reset in each `@BeforeEach` method so that state does not leak between tests. Integration tests use custom map strings rather than the production level to control the exact positions of walls, items, and entities.
- Reflection is used where necessary to access private inner classes (VisualState, UIElement, UIButton) and private fields, but only when no public API exists. Public methods are tested through their public interface wherever possible.
- Swing rendering tests run in headless mode (`java.awt.headless=true`) and paint into an off-screen BufferedImage, ensuring the full test suite can run in CI without a display server.

4.2 Line and Branch Coverage

Coverage was measured using JaCoCo 0.8.12 and is generated automatically during `mvn clean test`. The HTML report is available at `target/site/jacoco/index.html`. The table below summarises per-class line and branch coverage results.

Class	Instructions	Branches	Lines	Methods
BankHeist	0%	n/a	0%	0%
Board	97%	81%	97%	98%
BoardFactory	100%	100%	100%	100%
CashStack	100%	n/a	100%	100%
Cell	100%	100%	100%	100%
DiamondSpawner	73%	0%	69%	83%
GlueTrap	100%	n/a	100%	100%
Guard	100%	85%	100%	100%
InkDart	100%	n/a	100%	100%
Interactable	100%	n/a	100%	100%
Interface	93%	79%	95%	82%
ItemFactory	95%	100%	92%	50%
MovingEntity	91%	67%	91%	80%
PlaytimeManager	100%	100%	100%	100%
Position	92%	88%	89%	88%
RareDiamond	100%	n/a	100%	100%
Thief	100%	100%	100%	100%
Total	**94%**	**81%**	**94%**	**90%**

215 tests total. All tests pass. 11 of 17 classes reach 100% line coverage. The project-wide totals are 94% instruction coverage, 81% branch coverage, and 94% line coverage.

4.3 Uncovered Code and Justification

The following classes or code segments are not fully covered, along with the reason each gap exists:

- **BankHeist (0%)** – Contains only the `main()` entry point which creates the JFrame, adds the Board and Interface panels, and calls `setVisible(true)`. This bootstraps the JVM window and cannot be tested in-process without launching a live GUI.
- **DiamondSpawner (73% instructions, 0% branches)** – Uses Swing timers for periodic spawn and despawn logic. The timer callback branches (spawn vs despawn) depend on real elapsed time and are not directly unit tested. The spawner is exercised indirectly through the Board's diamond management methods which are fully tested.
- **System.exit(0) lambdas** – Three instructions across the Interface class are the `System.exit(0)` calls inside exit button lambdas. Executing these would terminate the JVM and the test runner. The gap is documented and intentional.
- **UIElement FitMode branch gaps** – Four UIElement branches correspond to the `iw <= 0 || ih <= 0` early-return guard inside dims(). This fires only when an image reports invalid dimensions. Since BufferedImage always returns valid sizes and production images load via ImageIcon, this path is unreachable in practice.

- **Interface mouse listener branches** – Two MouseAdapter branches represent the fall-through case where no button matches the click point. Two MouseMotionAdapter branches correspond to the hover-while-pressed guard. Both are no-op paths by design.
- **Board remaining branches** – The uncovered Board branches are primarily inside the gameMovement timer's guard AI block (guard angle computation, guard-thief collision, guard stepping on non-glue items) and the loadBoard map parser's character switch arms. These paths are exercised by the integration tests but are not isolated in unit tests.
- **ItemFactory (50% methods)** – The uncovered method is the implicit default constructor which is never called directly. All factory logic is tested through BoardFactory and the item creation tests.

5 Findings & Improvements

The following changes and observations resulted from writing and running the test suite.

5.1 Production Code Changes

- **MoveThief refactor** – The original implementation of moveThief was checking the global keyboard state (inputRow and inputCol) instead of the method parameters (dRow, dCol). This made it impossible to drive the thief from a test environment. The method was refactored to use its parameters directly.
- **Added resetInstance()** – A resetInstance() method was added to the Board class to reset the singleton instance between tests. Without this, test isolation was impossible because the singleton persisted across test methods.
- **Added accessor methods** – The following package-private accessor methods were added to expose internal state for testing without changing the public API:
  - getStaticMoneyCount
  - getInteractableCells
  - getThiefRow
  - getThiefCol
  - getGuardStuckCount

5.2 Bugs and Observations

- **Guard glue-trap timing** – Testing revealed that the board evaluates the guard's position after guard.update() moves it, not before. Placing the glue trap at the guard's starting cell produced no freeze. Placing it at the destination cell (one step ahead) correctly triggered a 3-tick freeze. This clarified the order of operations in the gameMovement timer.

- **GlueTrap timer dependency** – During the development of GlueTrapIntegrationTest, we found that the Guard's movement is tied directly to the game's UI timer. In our testing environment this timer does not run, which prevented the Guard from moving into the trap. The solution was to stop the timer and fire manual ActionEvent ticks to simulate the game loop.
- **Guard-catch priority** – Testing confirmed that the game correctly prioritizes the guard-catch loss condition over the exit win condition when both occur in the same movement tick. No implementation changes were required.
- **guardStuckCount requires a live guard** – The decrement logic is guarded by `if (guard != null && thief != null)`. Tests that used the default setUp map (no G character) saw the count never change. Integration tests that declare a G in the map string produce a live guard instance and exercise the decrement path correctly.
- **JaCoCo + Java 25 warnings** – Running JaCoCo 0.8.12 against a Java 25 toolchain emits IllegalClassFormatException warnings for JDK-internal classes (class file version 69). These are harmless and do not affect the coverage report. Upgrading to JaCoCo 0.8.13 or later will suppress them.